Retail Demand Forecasting – Amazon Case Study

- Abdelaziz Amr Mettwally

Import Libraries

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Load Data

In [2]:
```
df = pd.read_csv("../data/sales.csv")
df.head()
```

Out[2]:

	id	date	store_nbr	family	sales	onpromotion
0	0	2013-01-01	1	AUTOMOTIVE	0.0	0
1	1	2013-01-01	1	BABY CARE	0.0	0
2	2	2013-01-01	1	BEAUTY	0.0	0
3	3	2013-01-01	1	BEVERAGES	0.0	0
4	4	2013-01-01	1	BOOKS	0.0	0

In [3]:
```
df.shape
```

Out[3]:
```
(3000888, 6)
```

Check Available Product Families

In [4]:
```
df['family'].unique()
```

Out[4]:
```
<StringArray>
[                'AUTOMOTIVE',                 'BABY CARE',
                    'BEAUTY',                 'BEVERAGES',
                     'BOOKS',               'BREAD/BAKERY',
                'CELEBRATION',                  'CLEANING',
                     'DAIRY',                     'DELI',
                     'EGGS',               'FROZEN FOODS',
                 'GROCERY I',                'GROCERY II',
                 'HARDWARE',         'HOME AND KITCHEN I',
        'HOME AND KITCHEN II',            'HOME APPLIANCES',
                 'HOME CARE',                 'LADIESWEAR',
            'LAWN AND GARDEN',                  'LINGERIE',
           'LIQUOR,WINE,BEER',                 'MAGAZINES',
                     'MEATS',              'PERSONAL CARE',
               'PET SUPPLIES',    'PLAYERS AND ELECTRONICS',
                    'POULTRY',             'PREPARED FOODS',
                    'PRODUCE', 'SCHOOL AND OFFICE SUPPLIES',
                    'SEAFOOD']
Length: 33, dtype: str
```

Filter Data for Store 1 and BEVERAGES

In [5]:
```
df_filtered = df[df["store_nbr"] == 1]
df_filtered = df_filtered[df_filtered["family"] == "BEVERAGES"]

df_filtered.head()
```

Out[5]:

	id	date	store_nbr	family	sales	onpromotion
3	3	2013-01-01	1	BEVERAGES	0.0	0
1785	1785	2013-01-02	1	BEVERAGES	1091.0	0
3567	3567	2013-01-03	1	BEVERAGES	919.0	0
5349	5349	2013-01-04	1	BEVERAGES	953.0	0
7131	7131	2013-01-05	1	BEVERAGES	1160.0	0

In [6]:
```
df_filtered.shape
```

Out[6]:
```
(1684, 6)
```

Convert Date Column and Sort Data

In [7]:
```
df_filtered['date'] = pd.to_datetime(df_filtered['date'])
df_filtered = df_filtered.sort_values('date')
```

Aggregate Daily Sales

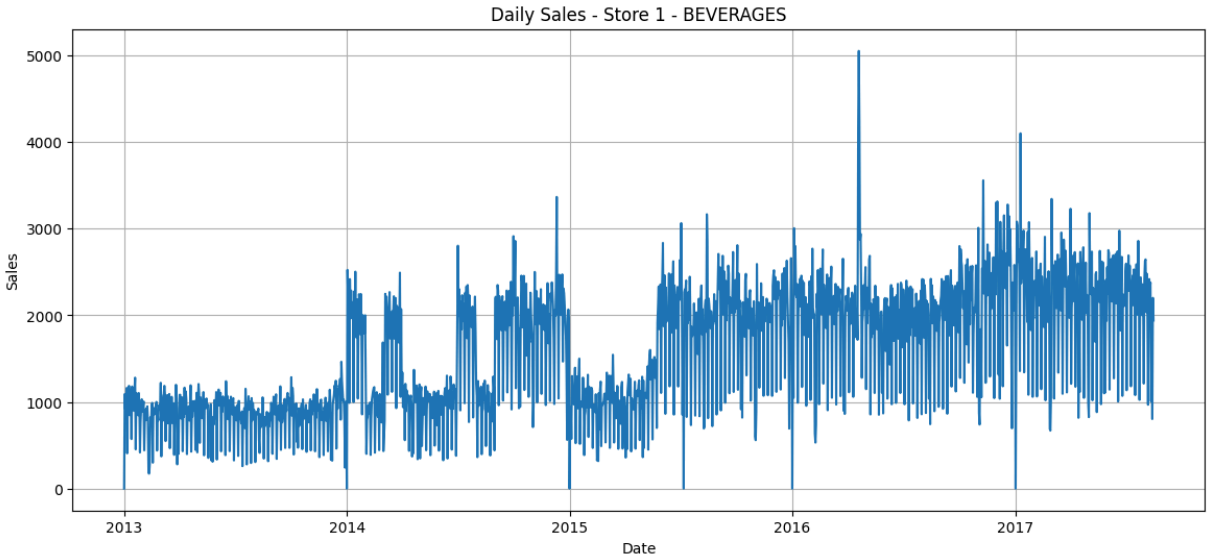
In [8]:
```
df_daily = df_filtered.groupby('date')['sales'].sum().reset_index()
df_daily = df_daily.set_index('date')

df_daily.head()
```

Out[8]:

date	sales
2013-01-01	0.0
2013-01-02	1091.0
2013-01-03	919.0
2013-01-04	953.0
2013-01-05	1160.0

In [9]:
```
plt.figure(figsize=(14,6))
plt.plot(df_daily.index, df_daily['sales'])
plt.title("Daily Sales - Store 1 - BEVERAGES")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.grid(True)
plt.show()
```



Feature Engineering

Lag Features

In [10]:
```
df_daily['lag_1'] = df_daily['sales'].shift(1)
df_daily['lag_7'] = df_daily['sales'].shift(7)
df_daily['lag_14'] = df_daily['sales'].shift(14)
```

```
df_daily.head(10)
```

Out[10]:

date	sales	lag_1	lag_7	lag_14
2013-01-01	0.0	NaN	NaN	NaN
2013-01-02	1091.0	0.0	NaN	NaN
2013-01-03	919.0	1091.0	NaN	NaN
2013-01-04	953.0	919.0	NaN	NaN
2013-01-05	1160.0	953.0	NaN	NaN
2013-01-06	407.0	1160.0	NaN	NaN
2013-01-07	1158.0	407.0	NaN	NaN
2013-01-08	1029.0	1158.0	0.0	NaN
2013-01-09	1186.0	1029.0	1091.0	NaN
2013-01-10	847.0	1186.0	919.0	NaN

Rolling Features

In [11]:

```
df_daily['rolling_mean_7'] = df_daily['sales'].rolling(window=7).mean()
df_daily['rolling_std_7'] = df_daily['sales'].rolling(window=7).std()

df_daily.head(10)
```

Out[11]:

date	sales	lag_1	lag_7	lag_14	rolling_mean_7	rolling_std_7
2013-01-01	0.0	NaN	NaN	NaN	NaN	NaN
2013-01-02	1091.0	0.0	NaN	NaN	NaN	NaN
2013-01-03	919.0	1091.0	NaN	NaN	NaN	NaN
2013-01-04	953.0	919.0	NaN	NaN	NaN	NaN
2013-01-05	1160.0	953.0	NaN	NaN	NaN	NaN
2013-01-06	407.0	1160.0	NaN	NaN	NaN	NaN
2013-01-07	1158.0	407.0	NaN	NaN	812.571429	442.221233
2013-01-08	1029.0	1158.0	0.0	NaN	959.571429	260.980112
2013-01-09	1186.0	1029.0	1091.0	NaN	973.142857	271.222804
2013-01-10	847.0	1186.0	919.0	NaN	962.857143	274.957832

Date-Based Features

In [12]:
```
df_daily['day_of_week'] = df_daily.index.dayofweek
df_daily['month'] = df_daily.index.month
df_daily['day'] = df_daily.index.day
```

In [13]:
```
df_daily = df_daily.dropna()
df_daily.shape
```

Out[13]:
```
(1670, 9)
```

Prepare Features and Target

In [14]:
```
ts = df_daily['sales'].asfreq('D')
ts = ts.interpolate()

ts.head()
```

Out[14]:
```
date
2013-01-15    1149.0
2013-01-16    1043.0
2013-01-17     898.0
2013-01-18    1130.0
2013-01-19    1280.0
Freq: D, Name: sales, dtype: float64
```

In [15]:
```
X = df_daily.drop('sales', axis=1)
y = df_daily['sales']

X.head()
```

Out[15]:

date	lag_1	lag_7	lag_14	rolling_mean_7	rolling_std_7	day_of_week	month	day
2013-01-15	1178.0	1029.0	0.0	1015.428571	229.109040	1	1	15
2013-01-16	1149.0	1186.0	1091.0	995.000000	217.443479	2	1	16
2013-01-17	1043.0	847.0	919.0	1002.285714	212.455261	3	1	17
2013-01-18	898.0	1033.0	953.0	1016.142857	217.886384	4	1	18
2013-01-19	1130.0	1143.0	1160.0	1035.714286	236.535208	5	1	19

Train-Test Split

In [16]:
```
split_index = int(len(df_daily) * 0.8)

X_train = X.iloc[:split_index]
X_test = X.iloc[split_index:]

y_train = y.iloc[:split_index]
y_test = y.iloc[split_index:]

X_train.shape, X_test.shape, y_train.shape, y_test.shape
```

Out[16]:
```
((1336, 8), (334, 8), (1336,), (334,))
```

Random Forest Model

In [17]:
```
from sklearn.ensemble import RandomForestRegressor

model_rf = RandomForestRegressor(
    n_estimators=200,
    max_depth=10,
    random_state=42
)

model_rf.fit(X_train, y_train)
```

Out[17]:

▾ RandomForestRegressor ⓘ ?

▸ Parameters

In [18]:
```
y_pred_rf = model_rf.predict(X_test)

y_pred_rf[:10]
```

Out[18]:
```
array([2077.56823458, 2243.34332808, 2220.09211343, 1023.21563303,
       2206.07088471, 2136.78402718, 2071.02499354, 2011.31297856,
       2298.88549829, 2237.25198992])
```

Random Forest Evaluation

In [19]:
```
from sklearn.metrics import mean_absolute_error, mean_squared_error

mae_rf = mean_absolute_error(y_test, y_pred_rf)
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))

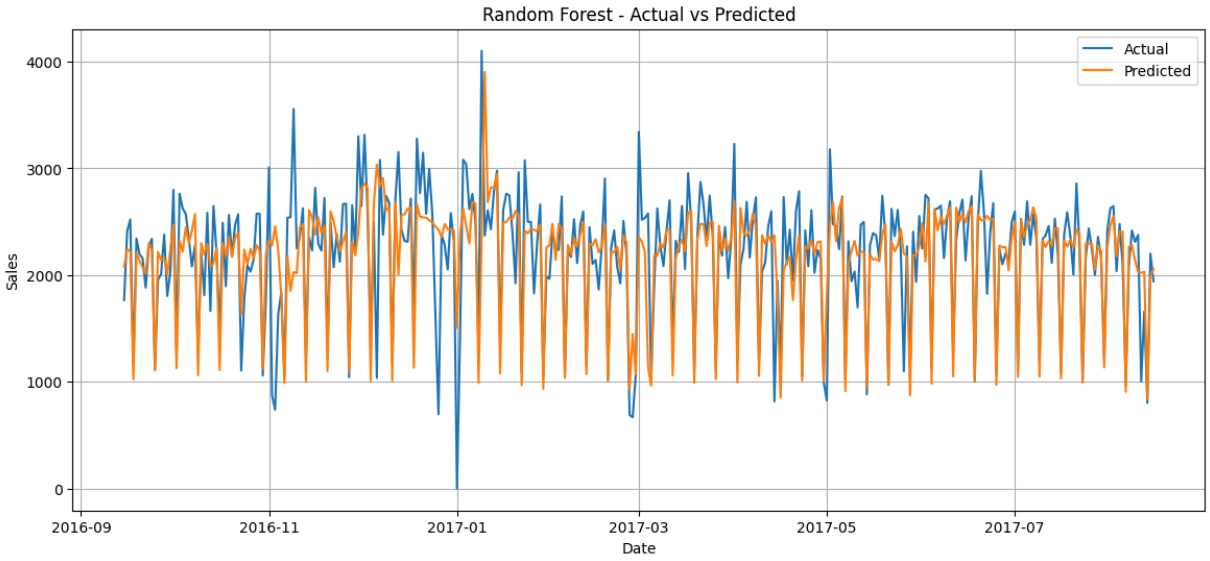
mae_rf, rmse_rf
```

Out[19]:
```
(264.68640965390887, np.float64(406.2755937821694))
```

In [20]:
```
plt.figure(figsize=(14,6))
plt.plot(y_test.index, y_test.values, label='Actual')
plt.plot(y_test.index, y_pred_rf, label='Predicted')
```

```
plt.title('Random Forest - Actual vs Predicted')
plt.xlabel('Date')
plt.ylabel('Sales')

plt.legend()
plt.grid(True)
plt.show()
```



In [21]:
```
feature_importance_rf = pd.Series(model_rf.feature_importances_, index=X.columns)
feature_importance_rf.sort_values(ascending=False)
```

Out[21]:
```
lag_7             0.680565
rolling_mean_7    0.102417
rolling_std_7     0.071835
day_of_week       0.054058
lag_1             0.036706
lag_14            0.032682
day               0.017235
month             0.004501
dtype: float64
```

XGBoost Model

In [22]:
```
from xgboost import XGBRegressor

model_xgb = XGBRegressor(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=5,
    random_state=42
)

model_xgb.fit(X_train, y_train)
```

Out[22]:
```
▾ XGBRegressor
▸ Parameters
```

In [23]:
```
y_pred_xgb = model_xgb.predict(X_test)

y_pred_xgb[:10]
```

Out[23]:
```
array([1901.2272 , 2224.383  , 2100.5615 , 1018.90765, 2140.0566 ,
       2027.1901 , 2146.4158 , 1930.8213 , 2345.6868 , 2319.5627 ],
      dtype=float32)
```

Initial XGBoost Evaluation

In [24]:
```
mae_xgb = mean_absolute_error(y_test, y_pred_xgb)
rmse_xgb = np.sqrt(mean_squared_error(y_test, y_pred_xgb))

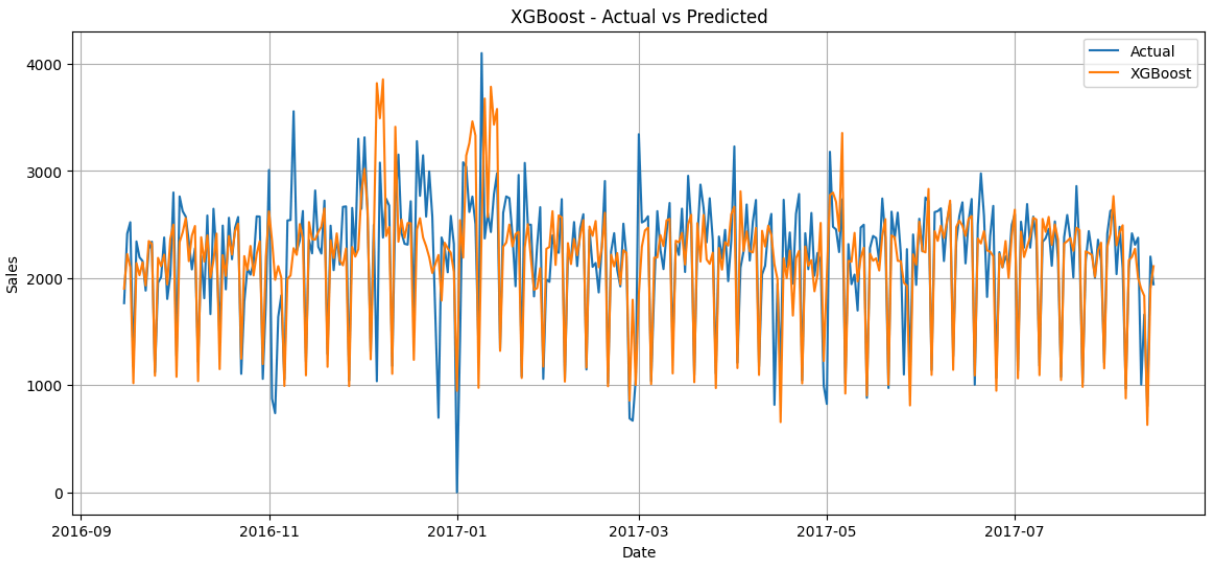
mae_xgb, rmse_xgb
```

Out[24]:
```
(278.3002728673512, np.float64(427.1775463023519))
```

In [25]:
```
plt.figure(figsize=(14,6))
plt.plot(y_test.index, y_test.values, label='Actual')
plt.plot(y_test.index, y_pred_xgb, label='XGBoost')

plt.title('XGBoost - Actual vs Predicted')
plt.xlabel('Date')
plt.ylabel('Sales')

plt.legend()
plt.grid(True)
plt.show()
```



Improve Feature Engineering

In [26]:
```
df_daily['week_of_year'] = df_daily.index.isocalendar().week.astype(int)
df_daily['is_weekend'] = (df_daily['day_of_week'] >= 5).astype(int)

df_daily['rolling_mean_14'] = df_daily['sales'].rolling(14).mean()
df_daily['rolling_std_14'] = df_daily['sales'].rolling(14).std()

df_daily = df_daily.dropna()
```

Rebuild Features and Split Data

In [27]:
```
X = df_daily.drop('sales', axis=1)
y = df_daily['sales']
```

In [28]:
```
split_index = int(len(df_daily) * 0.8)

X_train = X.iloc[:split_index]
X_test = X.iloc[split_index:]

y_train = y.iloc[:split_index]
y_test = y.iloc[split_index:]
```

Optimized XGBoost Model

In [29]:
```
model_xgb = XGBRegressor(
    n_estimators=1000,
    learning_rate=0.02,
    max_depth=10,
    subsample=0.9,
    colsample_bytree=0.9,
    random_state=42
)

model_xgb.fit(X_train, y_train)
```

Out[29]:
```
▾ XGBRegressor (i) (?)

▸ Parameters
```

In [30]:
```
y_pred_xgb = model_xgb.predict(X_test)
```

Optimized XGBoost Evaluation

In [31]:
```
mae_xgb = mean_absolute_error(y_test, y_pred_xgb)
rmse_xgb = np.sqrt(mean_squared_error(y_test, y_pred_xgb))

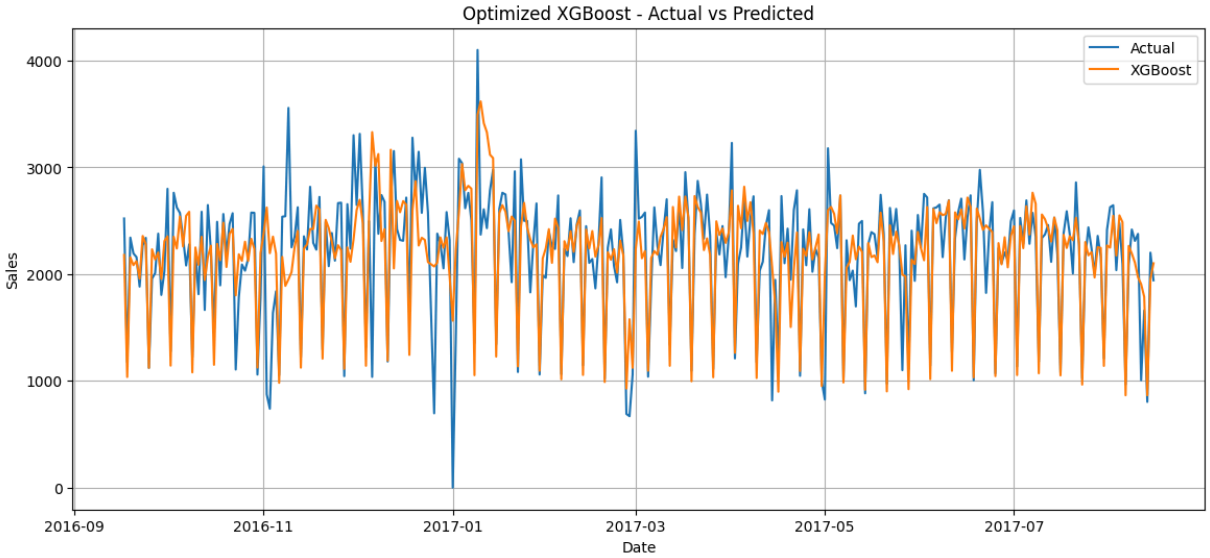
mae_xgb, rmse_xgb
```

Out[31]: (256.97167270154836, np.float64(393.4704461749039))

In [32]:
```
plt.figure(figsize=(14,6))
plt.plot(y_test.index, y_test.values, label='Actual')
plt.plot(y_test.index, y_pred_xgb, label='XGBoost')

plt.title('Optimized XGBoost - Actual vs Predicted')
plt.xlabel('Date')
plt.ylabel('Sales')

plt.legend()
plt.grid(True)
plt.show()
```



In [33]:
```
feature_importance = pd.Series(model_xgb.feature_importances_, index=X.columns)
feature_importance.sort_values(ascending=False)
```

Out[33]:
```
is_weekend         0.294929
lag_7              0.285678
day_of_week        0.131777
rolling_mean_7     0.121146
rolling_std_7      0.034807
rolling_std_14     0.032794
day                0.022444
lag_14             0.020477
week_of_year       0.019598
rolling_mean_14    0.018646
lag_1              0.010671
month              0.007032
dtype: float32
```

Time Series Cross-Validation

In [34]:
```
from sklearn.model_selection import TimeSeriesSplit

tscv = TimeSeriesSplit(n_splits=5)
```

In [35]:
```
mae_scores = []
rmse_scores = []

for train_index, test_index in tscv.split(X):
    X_train_cv, X_test_cv = X.iloc[train_index], X.iloc[test_index]
    y_train_cv, y_test_cv = y.iloc[train_index], y.iloc[test_index]

    model = XGBRegressor(
        n_estimators=1000,
        learning_rate=0.02,
        max_depth=10,
        subsample=0.9,
        colsample_bytree=0.9,
        random_state=42
    )

    model.fit(X_train_cv, y_train_cv)
    y_pred_cv = model.predict(X_test_cv)

    mae = mean_absolute_error(y_test_cv, y_pred_cv)
    rmse = np.sqrt(mean_squared_error(y_test_cv, y_pred_cv))

    mae_scores.append(mae)
    rmse_scores.append(rmse)

print("MAE:", np.mean(mae_scores))
print("RMSE:", np.mean(rmse_scores))
```

```
MAE: 283.2849052318628
RMSE: 434.9205143973239
```

SARIMA Model

In [36]:
```
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.stattools import adfuller
```

Prepare Time Series

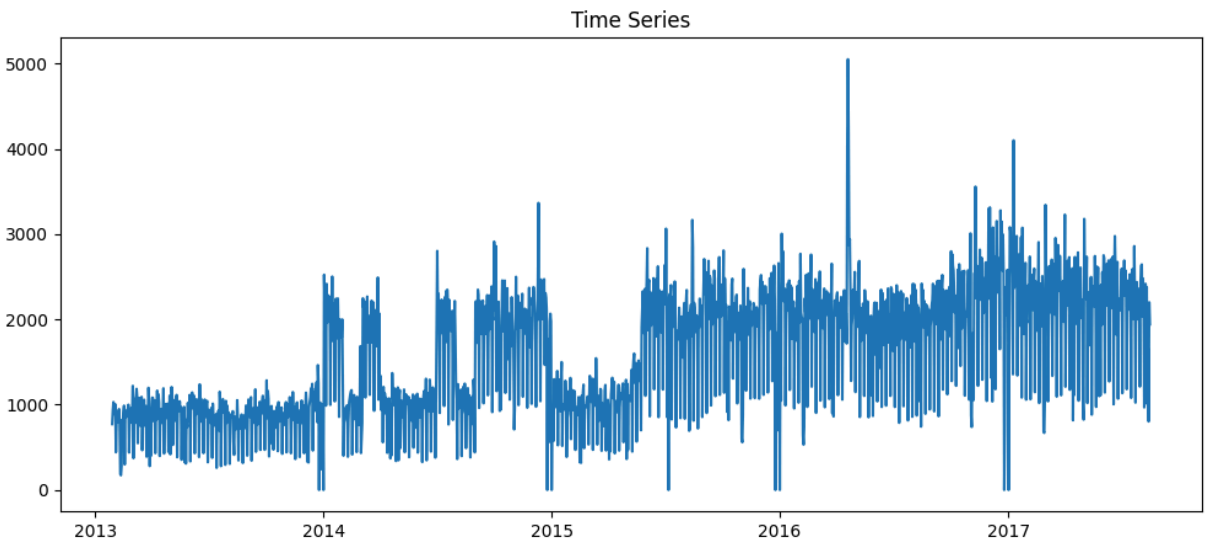
In [37]:
```
ts = df_daily['sales'].asfreq('D')
ts = ts.fillna(0)

ts.head()
```

Out[37]:
```
date
2013-01-28     772.0
2013-01-29     932.0
2013-01-30    1031.0
2013-01-31     889.0
2013-02-01     941.0
Freq: D, Name: sales, dtype: float64
```

In [38]:
```
plt.figure(figsize=(12,5))
plt.plot(ts)
```

```
plt.title("Time Series")
plt.show()
```



Stationarity Test

In [39]:
```
result = adfuller(ts)

print("ADF Statistic:", result[0])
print("p-value:", result[1])
```

```
ADF Statistic: -3.2804965200018095
p-value: 0.01577311043245655
```

Fit SARIMA Model

In [40]:
```
model = SARIMAX(
    ts,
    order=(1,0,1),
    seasonal_order=(1,1,1,7)
)

model_fit = model.fit()
print(model_fit.summary())
```

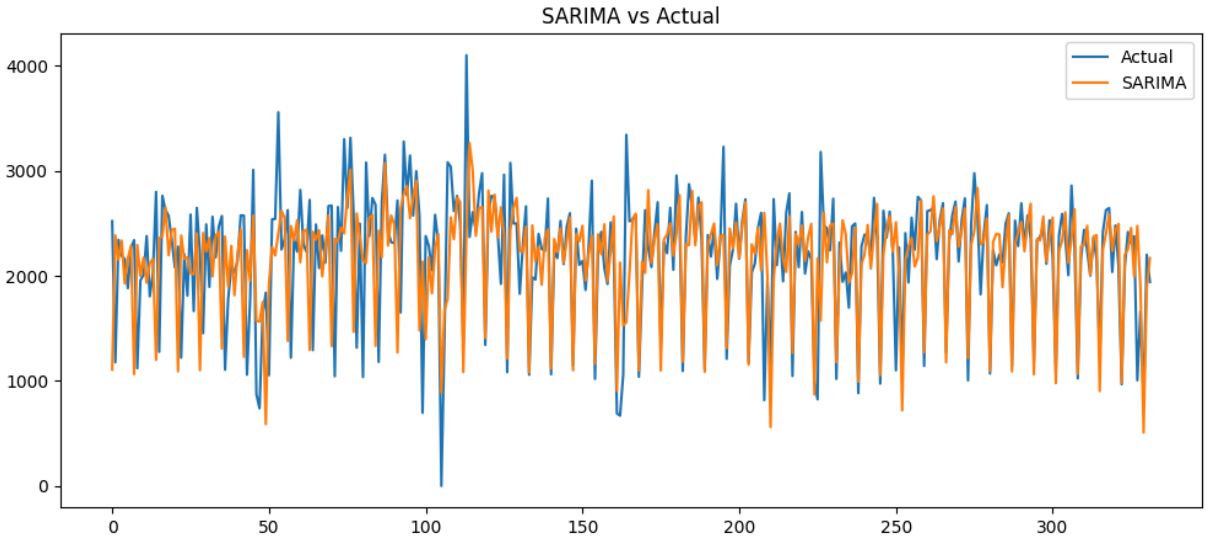
```
                                SARIMAX Results
==========================================================================================
Dep. Variable:                             sales   No. Observations:
1661
Model:             SARIMAX(1, 0, 1)x(1, 1, 1, 7)   Log Likelihood              -1197
6.787
Date:                           Thu, 23 Apr 2026   AIC                           2396
3.575
Time:                                  02:33:26   BIC                           2399
0.630
Sample:                              01-28-2013   HQIC                          2397
3.605
                                   - 08-15-2017
Covariance Type:                             opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1          0.8742      0.016     53.783      0.000       0.842       0.906
ma.L1         -0.4795      0.022    -21.987      0.000      -0.522      -0.437
ar.S.L7        0.1167      0.016      7.120      0.000       0.085       0.149
ma.S.L7       -0.9448      0.009   -105.444      0.000      -0.962      -0.927
sigma2      1.131e+05   1715.228     65.950      0.000     1.1e+05    1.16e+05
==========================================================================================
Ljung-Box (L1) (Q):                   1.75   Jarque-Bera (JB):              6988.45
Prob(Q):                              0.19   Prob(JB):                         0.00
Heteroskedasticity (H):               2.66   Skew:                            -0.39
Prob(H) (two-sided):                  0.00   Kurtosis:                        13.04
==========================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-ste
p).
```

SARIMA Prediction

In [41]:
```
pred = model_fit.predict(start=len(ts)-len(y_test), end=len(ts)-1)
```

In [42]:
```
plt.figure(figsize=(12,5))
plt.plot(y_test.values, label="Actual")
plt.plot(pred.values, label="SARIMA")
plt.legend()
plt.title("SARIMA vs Actual")
plt.show()
```



SARIMA Evaluation

In [43]:
```
mae_sarima = mean_absolute_error(y_test, pred)
rmse_sarima = np.sqrt(mean_squared_error(y_test, pred))

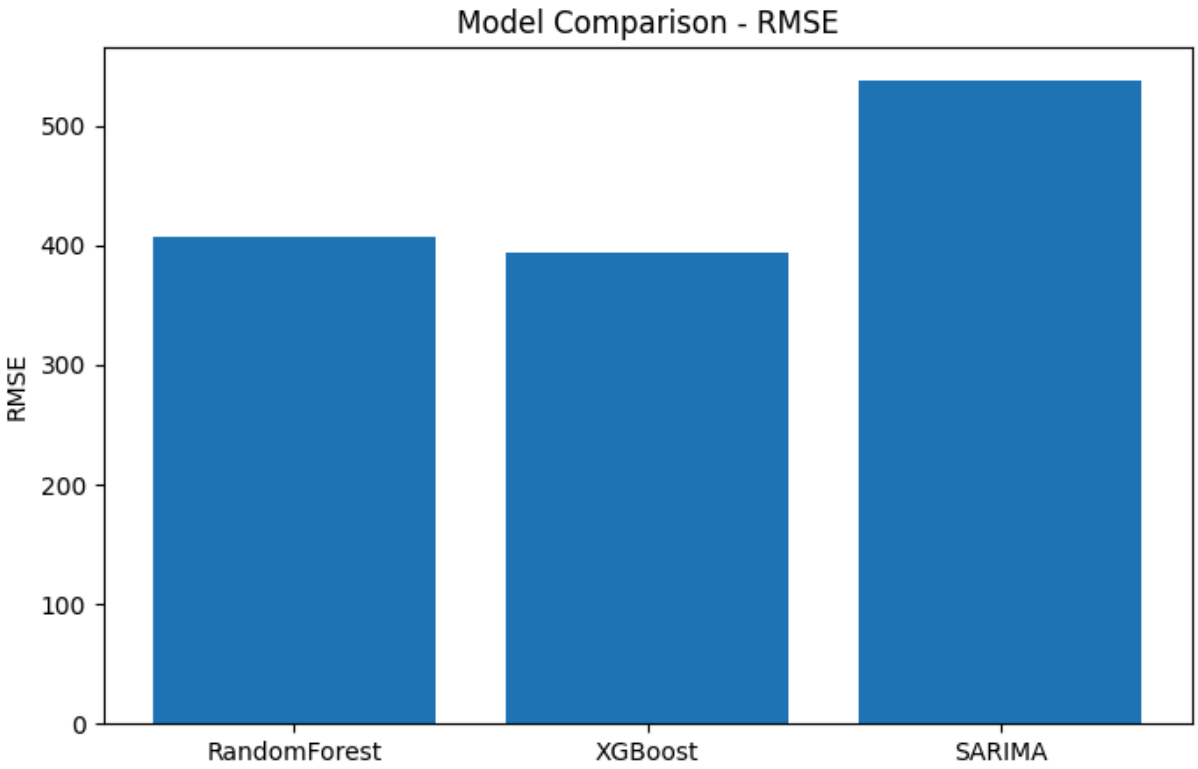
mae_sarima, rmse_sarima
```

Out[43]:
```
(358.0201620261184, np.float64(538.1247151760633))
```

Model Comparison

In [44]:
```
models = ['RandomForest', 'XGBoost', 'SARIMA']
mae_vals = [mae_rf, mae_xgb, mae_sarima]
rmse_vals = [rmse_rf, rmse_xgb, rmse_sarima]

plt.figure(figsize=(8,5))
plt.bar(models, rmse_vals)
plt.title("Model Comparison - RMSE")
plt.ylabel("RMSE")
plt.show()
```



In [45]:
```
comparison_df = pd.DataFrame({
    'Model': models,
    'MAE': mae_vals,
    'RMSE': rmse_vals
})

comparison_df
```

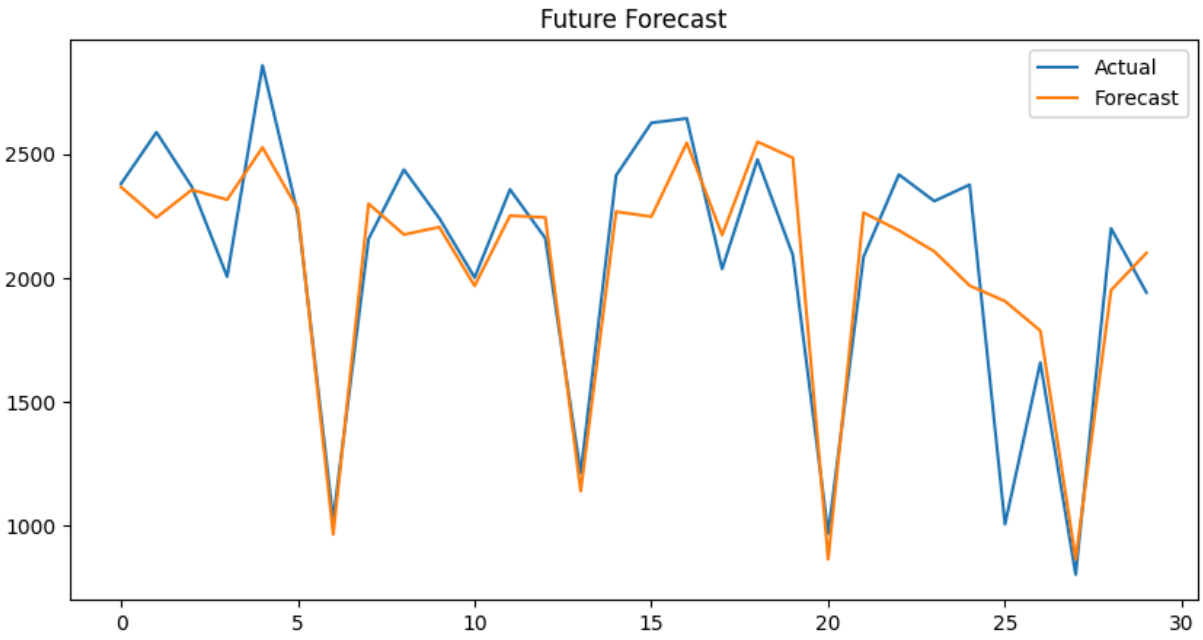
Out[45]:

	Model	MAE	RMSE
0	RandomForest	264.686410	406.275594
1	XGBoost	256.971673	393.470446
2	SARIMA	358.020162	538.124715

Future Forecast

In [46]:
```
future_pred = model_xgb.predict(X_test.tail(30))

plt.figure(figsize=(10,5))
plt.plot(y_test.tail(30).values, label="Actual")
plt.plot(future_pred, label="Forecast")
plt.legend()
plt.title("Future Forecast")
plt.show()
```



Why XGBoost Performed Best?

- Handles non-linearity better
- Uses boosting to reduce errors progressively
- Works well with engineered features such as lag variables and rolling statistics
- Captures complex retail demand patterns better than classical models

Final Conclusion

Among all tested models, XGBoost achieved the best performance with the lowest MAE and RMSE, significantly outperforming both Random Forest and SARIMA.

While SARIMA provided statistically valid results and captured basic temporal patterns, it struggled with the non-linear and highly variable nature of the sales data.

On the other hand, machine learning models, particularly XGBoost, demonstrated superior capability in capturing complex patterns through feature engineering, including lag variables and rolling statistics.

This highlights that for real-world retail demand forecasting, where data is often noisy and non-linear, machine learning approaches are generally more effective than classical time series models.

Business Insights

- Sales show strong weekly seasonality
- Beverage demand has noticeable volatility over time

- Rolling and lag-based features provide strong predictive power
- Machine learning models are more suitable for complex retail forecasting scenarios